arXiv:2605.11036v2 [cs.CR] 19 Oct 2026

Sequential Behavioral Watermarking for LLM Agents

[1]Hyeseon An, [2]Shinwoo Park,* [1]Dongsu Kim, and [1]Yo-Sub Han†
Department of Computer Science
[1]Yonsei University
[2]University of Luxembourg
{hsan, 123mki, emmous}@yonsei.ac.kr
shinwoo.park@uni.lu

Abstract

LLM-based agents act through sequences of executable decisions, but their trajectories provide little evidence of which agent or policy produced them, making provenance, ownership, and unauthorized reuse difficult to establish from observed behavior alone. This motivates watermarking signals embedded directly into agent behavior rather than only into generated text, since text watermarking cannot capture the action-level decisions that define agent execution. Recent agent watermarking methods address this gap by moving the watermark from generated text to behavioral choices. However, by treating each action step as an independent trial, they overlook trajectory structure and become fragile when trajectories are perturbed, truncated, or observed without reliable alignment. We propose `SeqWM`, a sequential behavioral watermarking framework that embeds signals into history-conditioned transition patterns and verifies trajectories position-agnostically against random-key baselines. Experiments across diverse agent benchmarks and LLM backbones show that `SeqWM` consistently achieves reliable detection while preserving agent utility, and remains robust under trajectory corruption where round-indexed behavioral watermarks collapse.

1 Introduction

Large language models have moved beyond the role of text generators and now serve as the decision-making core of autonomous agents that plan, invoke tools, and act on external environments over long horizons. Systems such as Claude Code [3], OpenAI's Deep Research [25], and Microsoft Copilot [23] are no longer experimental: they are deployed into production workflows where the agent itself decides how to decompose tasks, which tools to call, and in what order. This shift has two consequences that are easy to state and increasingly hard to ignore. First, the locus of intellectual property has moved from human-designed orchestration logic to the agent's intrinsic decision-making capability, which is acquired through expensive post-training on curated trajectory data [35]. Second, the trajectories themselves have become valuable assets: producing a single high-quality SWE-Bench-style trajectory is reported to cost on the order of $100 [26], individual tool-use dialogues run roughly $8 each [19], and curating the 120-task Mind2Web 2 benchmark required over 1,000 hours of human labor [9]. Agents and the data that train them have become economic objects whose unauthorized replication is now both technically feasible and commercially attractive, while the infrastructure for verifying their provenance has not kept up.

Across these deployments, a single technical problem recurs: given an observed sequence of agent actions, can one verify which agent produced it? The same primitive underlies a range of downstream concerns, from detecting impersonation and coordinated manipulation on open social platforms [12,

*This work was carried out while the author was at Yonsei University.
†Corresponding author

13], to attributing automated content to its originating system, to defending proprietary agents from imitation attacks in which an adversary harvests trajectories through ordinary API access and trains a surrogate that recovers most of the victim's capability [35]. Trajectory datasets released for research face the parallel risk of being used downstream without attribution [22].

Why token-level watermarking does not transfer. The natural starting point is the rich literature on watermarking for LLM-generated text, which biases token-level sampling toward a keyed subset of the vocabulary [16, 5]. When applied to agents, however, this approach is mismatched with the unit at which agents produce, and the unit at which detectors observe, behavior. At generation time, a single high-level decision such as which tool to call is realized through many tokens, and the green/red bias on each individual token largely shapes syntactic surface form rather than the decision itself; the watermark signal is diluted before it has any meaningful effect on which action is taken. At verification time, the situation is worse: a survey of 29 commercial agentic platforms reports that 82.8% expose only the final action stream while concealing internal reasoning traces and intermediate tokens [35], so whatever residual signal had survived generation is largely discarded before reaching the detector. Training-time alternatives that embed the signal directly into model weights face their own obstacle, since most deployed agents are built on closed pre-trained APIs that cannot be modified. The signal, in short, is lost twice—once into the wrong substrate and once at the visibility boundary—which suggests that watermarking for agents must operate at the level of *behavioral* decisions rather than tokens.

Existing behavioral watermarks. Recent work has begun to watermark LLM agents at the behavioral level, either by biasing action choices, modifying trajectory data, or exploiting equivalent tool-execution paths [12, 22, 35, 13]. These approaches establish action-level provenance as a practical direction, but they do not fully exploit the sequential structure of agent behavior. In particular, the watermark evidence is still tied to individual decisions, triggered actions, or fixed execution choices, rather than to the conditional transition patterns that characterize how an agent acts over time.

`SeqWM` takes this sequential structure as the carrier of the watermark. Instead of seeding the signal by absolute round index, we condition it on a window of recent actions, making the watermark depend on local behavioral transitions. This removes the need for strict positional alignment and makes deletion or partial observation a local rather than global failure mode. We further add multi-channel redundancy to strengthen the surviving evidence, and use random-key calibration to obtain valid detection without assuming independent sliding-window indicators. Together, these choices shift behavioral watermarking from per-step action statistics to position-agnostic verification of sequential behavior.

Contributions. We make the following contributions:

- **Framing.** We recast agent behavioral watermarking as a sequential coding problem, and identify an asymmetry overlooked by prior work: while watermark generation is naturally chained through action history, verification need not reconstruct the chain and can instead aggregate local transition evidence position-agnostically.
- **Method.** We introduce `SeqWM`, a three-component framework consisting of history-conditioned multi-channel encoding, sliding-window detection, and random-key calibration. Together, these components embed watermark signals into conditional behavioral transitions while enabling robust verification under deletion, truncation, and partial observation.
- **Theory and analysis.** We analyze the robustness and statistical behavior of sequential behavioral watermarking, including the local effect of deletion under sliding-window detection and the finite-sample validity of random-key calibration without independence assumptions.
- **Empirical evaluation.** Across multiple agent benchmarks and open-weight LLMs, SeqWM consistently achieves reliable watermark detection while preserving agent utility, and remains substantially more robust to action-level corruption than round-indexed behavioral watermarking baselines.

2 Related Work

Watermarking for Large Language Models. LLM watermarking is most commonly studied at the token level, where a keyed pseudorandom rule biases generation toward a detectable subset of tokens [16]. Subsequent work has improved this paradigm through low-distortion or distribution-preserving sampling [1, 4, 18], large-scale deployed systems [5], and semantic watermarks designed to better survive paraphrasing [21, 11]. At the same time, recent studies have clarified the limits of token-level watermarking, including reliability under editing and length variation [17], watermark stealing [14], robustness–utility trade-offs [27], and watermark inheritance through distillation [2]. These methods provide provenance for generated text, but they do not directly apply to LLM agents, where the relevant evidence is often a discrete and partially observable action trajectory rather than the underlying token stream.

Watermarking for LLM Agents. Agentic systems interleave model decisions with environment observations and tool executions [40], so detectors may observe only external actions rather than internal tokens or reasoning traces. This has motivated behavior-level watermarks for LLM agents. AGENT GUIDE biases per-step behavior choices toward keyed subsets and detects the resulting marginal frequency shift with a statistical test [12], while AGENTMARK embeds multi-bit identifiers into behavior choices while preserving the elicited per-step distribution [13]. Other work targets trajectory or system-level provenance: ACTHOOK watermarks training trajectories via trigger-conditioned hook actions [22], and AGENTWM protects fine-tuned agents by biasing among semantically equivalent tool-execution paths under grey-box observation [35]. These approaches establish behavioral watermarking as a practical direction, but they largely treat evidence as per-step signals or structured payloads. In contrast, SeqWM embeds the signal in history-conditioned transition patterns and verifies trajectories with position-agnostic, random-key-calibrated detection.

3 Preliminaries

Sequential agent behavior. We consider an LLM agent that interacts with an environment over T steps. At step t, the agent observes a state and selects an action b_t from a finite candidate set $\mathcal{B}_t$ with $|\mathcal{B}_t| = A$.[3] Let $P_t \in \Delta(\mathcal{B}_t)$ denote the action distribution elicited from the agent at step t in the absence of any watermarking, and let P_t^{wm} denote its watermarked counterpart.

Keyed subset primitive. The encoder and the detector share a secret key $\mathcal{K}$. For any byte string x, both can derive a pseudorandom subset of $\mathcal{B}_t$ via

$$G_{\mathcal{K}}(x) \triangleq \mathrm{SampleSubset}\big(\mathcal{B}_t,\ \mathrm{HMAC}(\mathcal{K}, x),\ n\big), \tag{1}$$

where $n \le A$ is a fixed subset size. We treat $G_{\mathcal{K}}$ as a uniformly random n-subset of $\mathcal{B}_t$ that is independent across distinct contexts x, which is a standard idealization of HMAC's pseudorandomness.

Threat model. The detector receives an observed action sequence $\hat{b}_{1:T'}$ together with the secret key $\mathcal{K}$ and must decide whether the observation originated from a watermarked source. The observation may be clean ($\hat{b}_{1:T'} = b_{1:T}$ and $T' = T$) or corrupted; in our analysis, corruption is modeled primarily as deletion of arbitrary positions, so that $T' \le T$.

Notation. Throughout, $A_t = |\mathcal{B}_t|$ denotes the candidate-set size at step t (and A when this size is constant), w the window length, m the number of channels, γ the bias strength, and $c_t = (b_{t-w}, \ldots, b_{t-1})$ the context window at step t. We write $\mathcal{B}_{g,t}^{(j)} \subseteq \mathcal{B}_t$ for the guided subset at step t on channel j, $\|$ for byte-string concatenation, and $\mathbb{1}[\cdot]$ for the indicator function.

4 Embedding Watermarks in Behavioral Transitions

We introduce `SeqWM`, a watermarking framework for the action sequences produced by LLM agents. The framework is built around three components, each addressing a distinct difficulty of watermarking

[3] We take the candidate-set size A to be constant across steps to simplify the exposition. When the size varies, the same construction applies with $A_t = |\mathcal{B}_t|$ and step-dependent null hit rate $p_{0,t} = n/A_t$.

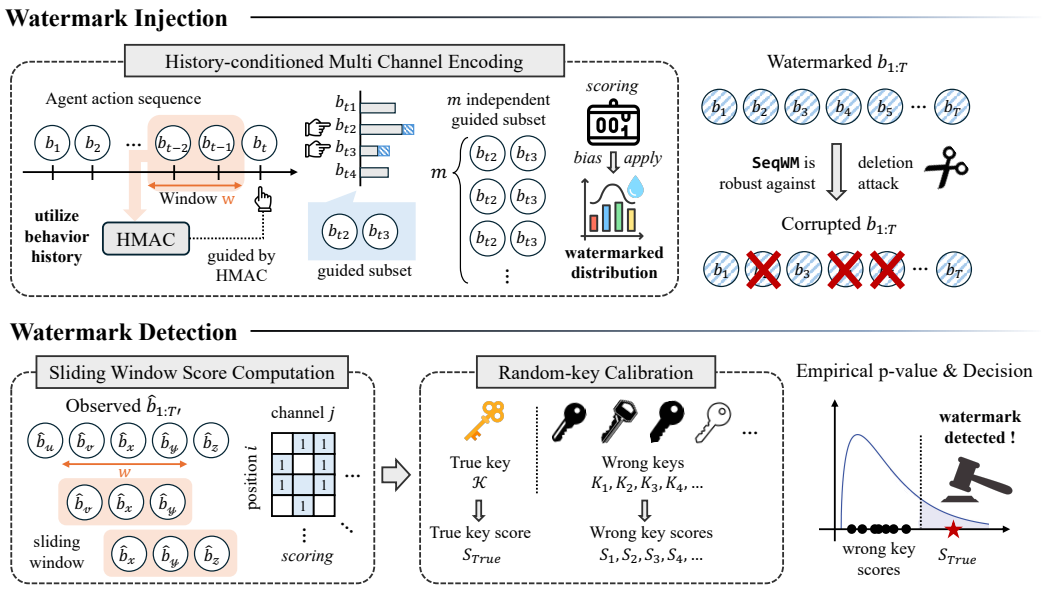


Figure 1: Overview of SeqWM. The injection step biases the agent's action distribution using multiple history-conditioned guided subsets, and the detection step slides over the observed sequence and calibrates the score against random keys to produce a p-value.

sequential agent behavior. First, a *history-conditioned multi-channel encoder* (Section 4.1) embeds the watermark signal into the agent's conditional transition distribution rather than its marginal action frequencies, and replicates the signal across m statistically independent channels. Second, a *sliding-window detector* (Section 4.2) discards any reliance on absolute step index, allowing detection on truncated, deleted, or partially observed sequences without alignment information. Third, a *random-key calibration* procedure (Section 4.3) constructs an empirical null distribution from the observed sequence itself, yielding valid statistical inference without assuming independence among detector indicators. We begin by formalizing the setting before describing each component in turn.

4.1 History-Conditioned Multi-Channel Encoding

From round indices to history-conditioned seeds. Round-indexed agent watermarks (e.g., *Agent Guide*) seed the guided subset directly with the absolute step index, $B_{g,t}^{\mathrm{AG}} = G_{\mathcal{K}}(t)$, treating each step as a statistically independent trial keyed by its position in the trajectory. Two limitations follow. The signal is encoded into the marginal action frequency, which is exactly what an adversary who profiles only first-order behavior can attack; and the encoder and decoder must share an absolute coordinate, so any deletion or truncation that perturbs the index permanently misaligns subsequent indicators with the keyed subsets.

SeqWM replaces the round index with a window of preceding actions:

$$B_{g,t} \;=\; G_{\mathcal{K}}(c_t), \qquad c_t = (b_{t-w}, \ldots, b_{t-1}). \tag{2}$$

This seemingly small change has two structural consequences. First, the watermark signal is now carried by the conditional transition distribution $P_t(\cdot \mid c_t)$: marginal action frequencies are not biased upward, only the dependence of b_t on its recent context is. An adversary who matches the marginal distribution therefore cannot remove the watermark without also disrupting transition statistics. Second, since the same window deterministically yields the same subset wherever it appears in the stream, the encoder and the decoder no longer require a shared notion of absolute position—a property we exploit in Section 4.2.

Multi-channel redundancy. We further replicate the watermark across m statistically independent channels by appending a channel index to the seed:

$$B_{g,t}^{(j)} \;=\; G_{\mathcal{K}}\big(c_t \,\|\, j\big), \qquad j = 1, \ldots, m. \tag{3}$$

The pseudorandomness of HMAC ensures that the m subsets behave as independent random n-subsets. We then summarize how strongly each candidate action is favored by the watermark via the multi-channel score

$$s_t(b) \;=\; \sum_{j=1}^{m} \mathbb{1}\big[b \in B_{g,t}^{(j)}\big] \;\in\; \{0, 1, \ldots, m\}, \tag{4}$$

which counts the number of channels in which b is a watermark target. The watermarked distribution applies an exponential tilt to the elicited distribution in proportion to this score:

$$P_t^{\mathrm{wm}}(b) \;=\; \frac{1}{Z_t}\, P_t(b)\, \exp\!\left(\gamma \cdot \frac{s_t(b)}{m}\right), \qquad Z_t \;=\; \sum_{b' \in \mathcal{B}_t} P_t(b')\, \exp\!\left(\gamma \cdot \frac{s_t(b')}{m}\right). \tag{5}$$

The exponential form is chosen for three reasons. It places the encoder in an exponential family with sufficient statistic $s_t(b)/m$ and natural parameter γ, which makes the subsequent KL analysis transparent. For small γ it reduces to a multiplicative bias, since $\exp(\gamma s/m) \approx 1 + \gamma s/m$. And it naturally amplifies actions that are favored by multiple channels relative to those favored by only one.

The encoder draws $b_t \sim P_t^{\mathrm{wm}}$ for $t > w$ and falls back to the unmodified P_t during the first w bootstrap steps when no full window is yet available. The procedure is summarized in Algorithm 1.

Detection power and utility cost. We characterize the encoder's per-step effect under a sum-score detector. Let $p_0 = n/A$ denote the probability that an action drawn from P_t falls into a single guided subset, which holds for any one channel under the pseudorandomness of $G_{\mathcal{K}}$. A first-order expansion of the exponential tilt around $\gamma = 0$, combined with channel independence, gives the corresponding probability under the watermarked distribution as

$$p_1 \;=\; p_0 + \frac{\gamma}{m}\, p_0(1 - p_0) + O(\gamma^2). \tag{6}$$

A sum-score detector that aggregates $N = (T - w)\, m$ indicators across steps and channels therefore produces an expected mean shift $\mu_1 - \mu_0 = N(p_1 - p_0) = (T - w)\, \gamma\, p_0(1 - p_0)$. Dividing by the binomial standard deviation $\sqrt{N\, p_0(1 - p_0)}$ yields the signal-to-noise ratio

$$\mathrm{SNR} \;=\; \sqrt{\frac{T - w}{m}} \cdot \gamma \sqrt{p_0(1 - p_0)}. \tag{7}$$

On the utility side, the second-order expansion of KL divergence in an exponential family, together with $s_t(b) \sim \mathrm{Binomial}(m, p_0)$ under uniform P_t, gives

$$D_{\mathrm{KL}}\big(P_t^{\mathrm{wm}} \,\|\, P_t\big) \;=\; \frac{\gamma^2}{2}\, \mathrm{Var}_{b \sim P_t}\left[\frac{s_t(b)}{m}\right] + O(\gamma^3) \;=\; \frac{\gamma^2}{2m}\, p_0(1 - p_0) + O(\gamma^3). \tag{8}$$

The mean-shift prediction in Eq. (7)'s numerator is directly testable: a watermarked sequence should produce $S - \mathbb{E}[S] \;\approx\; (T - w)\gamma p_0(1 - p_0)$. Table 1 confirms this across all three domains: the measured gap matches the first-order prediction within a small constant factor (ratios 0.91 to 1.42), with deviations consistent with the $O(\gamma^2)$ correction at $\gamma = 2.0$. Random-key calibration also yields an unbiased null estimator: under unwatermarked sequences, the measured gap is at most 1.74% of $\mathbb{E}[S]$ across all domains.

Table 1: Mean-shift validation. The measured score gap ΔS matches the first-order theoretical prediction ΔS_{pred} within a small constant factor across all three domains.

Dataset	ΔS_{pred}	ΔS	Ratio
ToolBench	193.0	216.8	1.12
ALFWorld	348.5	318.1	0.91
OASIS	43.8	62.4	1.42

Eqs. (7) and (8) jointly imply that the ratio $\mathrm{SNR}^2 / D_{\mathrm{KL}}$ is independent of m. Under a plain sum-score detector, redundant channels do not

improve the raw information-theoretic trade-off between detection power and distortion—a fact we state plainly because it is easy to obscure. The value of m instead lies on three orthogonal axes that the SNR/KL ratio does not capture. *(i) Robustness buffer.* Redundant channels provide multiple local indicators for each transition, so deletions remove evidence locally rather than causing the global misalignment suffered by round-indexed detectors. Under the normalized tilt in Eq. ((5)), however, the sum-score mean shift does not grow linearly with m; the robustness benefit is therefore a local redundancy effect rather than an information-theoretic gain for the plain sum-score detector. *(ii) Calibration concentration.* The relative standard deviation of the wrong-key score used in Section 4.3 scales as $\big((T-w)\,m\big)^{-1/2}$, sharpening the empirical null distribution and improving p-value resolution at modest sequence length and calibration size. *(iii) Detector extensibility.* The multi-valued score $s_t(b) \in \{0, \ldots, m\}$ admits weighted detectors and likelihood-ratio tests that strictly dominate the sum-score; the latter is the simplest, not the most powerful, use of the multi-channel structure, and the calibration procedure of Section 4.3 accommodates any such replacement without modification.

4.2 Sliding-Window Detection

Removing absolute position from detection. A round-indexed detector verifies, for each t, whether $b_t \in G_{\mathcal{K}}(t)$, and is therefore brittle: a single deletion shifts every downstream action by one index, so b_{t+1} is now compared against a subset that was never used to bias it, and the resulting noise propagates indefinitely. The history-conditioned encoder of Section 4.1 permits a structurally different detector that discards the absolute index entirely. Since the same window deterministically yields the same guided subsets under the encoder and the decoder, evidence of watermarking can be aggregated directly from every length-w substring of the observed sequence:

$$S(\mathcal{K}, \hat{b}_{1:T'}) \;=\; \sum_{i=1}^{T'-w} \sum_{j=1}^{m} \mathbb{1}\Big[\hat{b}_{i+w} \in G_{\mathcal{K}}\big(\hat{b}_{i:i+w-1} \,\|\, j\big)\Big]. \tag{9}$$

Each term examines whether the action immediately following a window matches the guided subset that the encoder would have used at *some* step where the same window appeared. The detector neither knows nor needs to know which step that was. Algorithm 2 summarizes the score computation.

An asymmetry between encoder and decoder. The construction exploits an asymmetry that does not arise in conventional per-step watermarks. The encoder is naturally chained: each step's bias depends on the immediately preceding w actions. The decoder, however, treats every length-$(w+1)$ substring of the observation as an independent piece of evidence and aggregates over them without following the chain. The situation is analogous to message authentication codes, where a single key permits sequential generation but allows verification of arbitrary fragments of a stream. This asymmetry is what enables robust detection on partial or rearranged observations.

Robustness to deletion. The structural difference between round-indexed and sliding-window detection manifests as a sharp asymptotic separation under deletion.

Theorem 4.1 (Deletion robustness)**.** *Let $b_{1:T}$ be a watermarked sequence and let $\hat{b}$ be obtained by deleting d positions chosen uniformly at random, with deletion fraction $\rho = d/T$. Assume the verifier can reconstruct the candidate action set used for each scored transition.*

We analyze the round-indexed baseline under the standard pseudorandom misalignment idealization: once a deletion shifts the detector's round index, an action generated under $G_{\mathcal{K}}(t')$ and tested against $G_{\mathcal{K}}(t)$ for $t \neq t'$ contributes only the null hit rate p_0.

(a) Round-indexed detection. *Let k^* denote the position of the first deletion. For all detector positions after the aligned prefix, i.e., $t \geq k^*$, the round-indexed indicator $\mathbb{1}[\hat{b}_t \in G_{\mathcal{K}}(t)]$ has expectation p_0. Consequently, the expected score gap of a round-indexed detector satisfies*

$$\mathbb{E}\big[S^{\mathrm{RI}}_{\mathrm{after}} - S^{\mathrm{RI}}_{\mathrm{null}}\big] \;=\; O(1/\rho), \tag{10}$$

which vanishes relative to the clean-sequence gap $\Theta(T)$ as $T \to \infty$ for any fixed $\rho > 0$.

(b) Sliding-window detection. *The score $S(\mathcal{K}, \hat{b})$ defined in Eq.* (9) *satisfies the deterministic additive bound*

$$S(\mathcal{K}, \hat{b}) \geq S(\mathcal{K}, b_{1:T}) - d(w+1)m. \tag{11}$$

Equivalently, the number of clean sliding-window indicators that can be invalidated by the deletions is at most $d(w+1)m$, independent of T.

The proof of (a) is a direct consequence of round-index misalignment: once position k^* is deleted, the encoder used $G_{\mathcal{K}}(t)$ to bias the action at original step t, but the detector now reads it as step $t-1$ and compares against $G_{\mathcal{K}}(t-1)$, which is an independent random subset under the pseudorandomness of G. The proof of (b) is the counting argument outlined above: each deleted position appears in at most w context windows and is the next-action of at most one window, so it invalidates at most $(w+1)\,m$ indicators in Eq. (9). Full proofs are deferred to Appendix D.

Theorem 4.1 captures the central design contrast of our method. For example, when $w = 3$, deleting a fraction ρ of the trajectory can invalidate at most $4\rho Tm$ clean sliding-window indicators. This contrasts with round-indexed detection, where the expected aligned prefix length is only $O(1/\rho)$. The bound is additive rather than a direct guarantee on the post-deletion detection gap; the empirical results in Section 5 measure the resulting detection power.

4.3 Random-Key Calibration

Why a closed-form null distribution does not apply. The natural inferential procedure for the score in Eq. (9) would be a z-test against a binomial null with variance $N\,p_0(1-p_0)$. This null is, however, invalid in our setting. Three sources of dependence couple the indicators that the test would treat as independent: actions b_t depend on b_{t-1} through the LLM's natural sequential structure; the watermark itself reuses the same guided subset whenever the same window reappears; and the overlapping length-$(w+1)$ substrings examined by the sliding-window detector share actions by construction. Each of these typically induces positive correlation, so a binomial-based test underestimates the variance of S under the null and inflates false-positive rates—an unacceptable failure mode for a statistical detector.

The wrong-key trick. We sidestep the closed-form null by constructing the null distribution *empirically from the observed sequence itself*. The construction rests on a single observation: re-evaluating the score in Eq. (9) on the same sequence $\hat{b}_{1:T'}$ but with a wrong key $\mathcal{K}_r \neq \mathcal{K}$ preserves *every* dependence in the sequence—both natural and watermark-induced—while erasing only the keyed alignment that produces the watermark signal. Whatever positive correlation the binomial null fails to account for is therefore present in equal measure in the wrong-key scores, and is automatically absorbed into the empirical null.

Concretely, we draw M independent random keys $\mathcal{K}_1, \ldots, \mathcal{K}_M$ from the key distribution, compute $S_r = S(\mathcal{K}_r, \hat{b}_{1:T'})$ under each, and report the empirical p-value

$$p \;=\; \frac{1 + \big|\{r : S_r \geq S_{\text{true}}\}\big|}{M+1}, \qquad S_{\text{true}} = S(\mathcal{K}, \hat{b}_{1:T'}), \tag{12}$$

declaring the sequence watermarked iff $p < \alpha$. The full procedure is given in Algorithm 3.

Validity and finite-sample guarantee. Under the null hypothesis that $\hat{b}_{1:T'}$ was not generated using $\mathcal{K}$, the sequence is statistically independent of every key, and the true and wrong keys are exchangeable draws from the key distribution. The vector $(S_{\text{true}}, S_1, \ldots, S_M)$ is therefore exchangeable, the rank of S_{true} is uniform on $\{1, \ldots, M+1\}$, and the p-value in Eq. (12) satisfies $\Pr[p \leq \alpha] \leq \alpha$ for every $\alpha \in (0, 1)$. The guarantee is finite-sample: it does not invoke any asymptotic regime in T', m, or M. Under the alternative, the true-key score S_{true} is systematically larger than the wrong-key scores, and the gap grows in γ and in sequence length.

Three properties of the resulting test. The wrong-key trick yields a test with several practical consequences. *(i) Distribution-free.* Any heterogeneity, skew, or temporal structure in P_t is mirrored in every S_r and cancels in the comparison; no assumption on the agent's action distribution enters the validity argument. *(ii) Robust to adversarial corruption.* If the sequence has been deleted, truncated, substituted, or partially observed, the same corrupted sequence is presented to every key; the corruption affects S_{true} and $\{S_r\}$ symmetrically and the p-value remains valid, though detection power may decrease as the watermark signal weakens. *(iii) Detector-agnostic.* Replacing Eq. (9) with any other test statistic computed from $(\mathcal{K}, \hat{b}_{1:T'})$—a likelihood ratio, a weighted aggregator, or a

Table 2: Watermark detection across three benchmarks and three LLMs. We report the detection z-score, p-value, and per-step hit rate (Hit%, undefined for unwatermarked and AgentMark). `SeqWM` is the only method that reliably rejects the null across all settings.

Model	Method	ToolBench			ALFWorld			OASIS		
		$z \uparrow$	$p \downarrow$	**Hit%**	$z \uparrow$	$p \downarrow$	**Hit%**	$z \uparrow$	$p \downarrow$	**Hit%**
Gemma	Unwatermarked	0.13	0.455	–	-0.93	0.830	–	-0.14	0.571	–
	AgentGuide	1.83	0.032	23%	0.89	0.197	17%	-1.06	0.861	49%
	AgentMark†	-0.71	0.761	–	-1.80	0.965	–	1.09	0.152	–
	`SeqWM` *(ours)*	**7.77**	**0.001**	**67%**	**9.45**	**0.001**	**66%**	**4.67**	**0.001**	**99%**
LLaMA	Unwatermarked	-0.21	0.580	–	-0.36	0.645	–	-0.24	0.609	–
	AgentGuide	-0.54	0.707	33%	-0.21	0.598	20%	-1.06	0.861	49%
	AgentMark†	-1.97	0.981	–	0.67	0.253	–	1.09	0.152	–
	`SeqWM` *(ours)*	**9.22**	**0.001**	**81%**	**10.18**	**0.001**	**75%**	**5.16**	**0.001**	**100%**
Qwen	Unwatermarked	0.08	0.466	–	0.21	0.416	–	0.15	0.455	–
	AgentGuide	-1.47	0.928	23%	-2.17	0.985	20%	1.01	0.165	44%
	AgentMark†	-1.14	0.879	–	1.34	0.087	–	-1.03	0.857	–
	`SeqWM` *(ours)*	**5.48**	**0.001**	**63%**	**11.44**	**0.001**	**79%**	**2.53**	**0.010**	**97%**

learned scorer—preserves validity, providing a principled route to stronger detectors that exploit the multi-channel structure of Section 4.1.

5 Experiments and Analysis

5.1 Experimental Setup

We evaluate our method on three agent benchmarks: ToolBench (200 queries from under a ReAct loop with at most 6 steps per episode), ALFWorld (50 tasks with maximum episode length of 50 and one-shot prompting), and OASIS Reddit (100 simulation steps with one user agent and three NPC agents). The agent LLM is one of LLaMA-3.2 3B Instruct, Gemma 4 E4B-it, or Qwen3-4B Instruct, decoded at temperature 0.7. Unless otherwise noted, watermark parameters are fixed at window size $w = 3$, $m = 8$ channels, guided-subset size $n = 3$ (with $n = 2$ on OASIS due to its smaller behavior set), floor $\delta = 0.2$, bias strength $\gamma = 2.0$, and a detection threshold of $\alpha = 0.01$ with $M = 1000$ random-key calibration trials. The γ-sensitivity ablation sweeps $\gamma \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ with m fixed at 8 on a 50-query ToolBench subset. All main rollouts run on NVIDIA A100 GPUs, with sanity checks and analysis sweeps offloaded to NVIDIA RTX PRO 6000 Blackwell GPUs on the same SLURM cluster. Full configuration details are deferred to Appendix E.

5.2 Detection Results

Table 2 shows that `SeqWM` consistently achieves strong watermark detection across all three benchmarks and all evaluated LLMs, while the baselines frequently fail to separate from the null.

A notable pattern is that strong sequence-level detection emerges even when the per-step hit rates are only moderately elevated. This indicates that the watermark is not concentrated in a small number of conspicuous actions, but instead accumulates gradually through many weak transition-level biases across the trajectory. The results therefore support the central premise of our method: sequential dependence itself can serve as a robust substrate for behavioral watermarking.

γ Sensitivity. We sweep $\gamma \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ on OASIS across three LLM backbones. We use OASIS as a challenging stress test because its trajectories are relatively short and its action vocabulary is small, leaving less room for watermark evidence to accumulate. Figure 2 shows that the multi-channel detector strengthens steadily as γ increases and reaches $p < 0.01$ from $\gamma = 2.0$

[3] † AgentMark is a multi-bit payload watermark, so the single-bit detection metrics (z, p) are not its native evaluation; we include them for cross-method comparability. Hit% is undefined under its differential bin encoding.

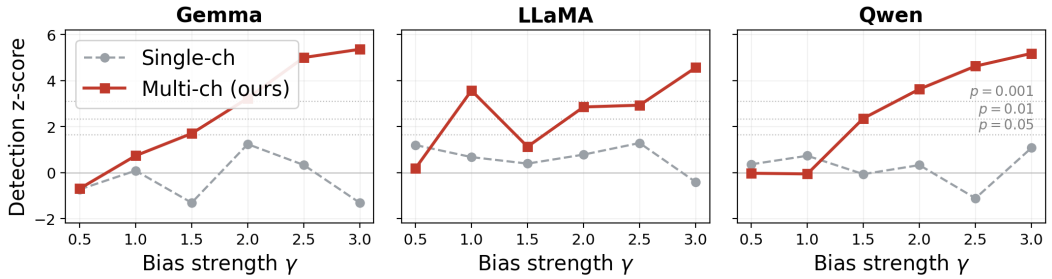


Figure 2: Detection z-score versus bias strength on OASIS Reddit across three LLM backbones; dotted lines mark standard significance thresholds. Multi-channel detection scales monotonically with bias and clears the strictest threshold on every backbone, while the single-channel ablation stays at the noise floor throughout.

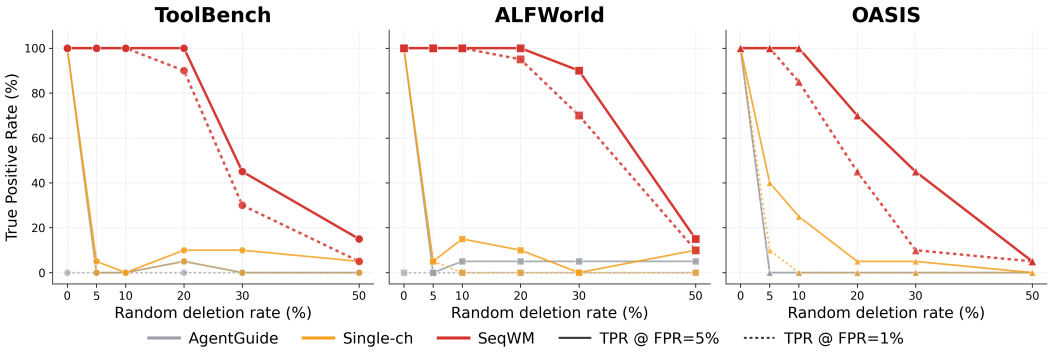


Figure 3: Detection TPR as a function of random action deletion rate across three domains (ToolBench, ALFWorld, OASIS). Colors distinguish methods and line styles distinguish FPR thresholds. Higher curves indicate stronger robustness to deletion attacks.

onward on all three backbones, suggesting a stable operating range rather than a model-specific tuning artifact. The single-channel ablation, however, remains near the null even at $\gamma = 3.0$. Thus, detection does not come from bias strength alone: reliable detection requires redundant evidence accumulated across channels together with the position-agnostic sequential detector. We therefore set $\gamma = 2.0$ as the default for the remaining experiments, choosing the smallest value that consistently yields reliable detection across backbones.

5.3 Robustness

We evaluate robustness under random action deletion, where a fraction $r \in \{0, 5, 10, 20, 30, 50\}\%$ of actions is removed before detection. This setting captures both partial logging and adversarial attempts to break trajectory-level provenance.

Figure 3 shows that round-indexed detection is brittle: AgentGuide collapses after small deletion rates because a single missing action misaligns all downstream keyed subsets. The single-channel variant avoids this global misalignment, but its score margin is too thin to survive substantial corruption.

In contrast, the full multi-channel SeqWM detector remains stable under moderate deletion across all three domains. Detection stays near-perfect through small deletion rates and degrades gradually rather than catastrophically as the deletion rate increases. This supports the design of SeqWM: history-conditioned seeding localizes the effect of deletion, while multi-channel redundancy preserves enough evidence from the surviving contextual transitions.

6 Conclusion

We presented SeqWM, a sequential behavioral watermarking framework for LLM agents that treats provenance as evidence accumulated from how actions evolve over time, rather than from marginal per-step action frequencies. By conditioning watermark signals on recent action history, distributing them across multiple behavioral channels, and verifying trajectories with position-agnostic sliding-window detection calibrated by random keys, SeqWM removes the need for fragile round alignment while preserving statistically valid detection. This design makes trajectory corruption a local source of evidence loss rather than a global detector failure: deletions, truncation, and partial observation affect only the surrounding transition windows instead of misaligning the remainder of the sequence. Across diverse agent benchmarks and LLM backbones, SeqWM achieves reliable watermark detection with preserved utility and remains robust under deletion rates that cause round-indexed and bit-encoding baselines to collapse. More broadly, our results suggest that sequential structure is not merely a nuisance in agent watermarking, but a powerful carrier of provenance signals; extending this perspective to stronger adaptive attacks, substitutions, and semantic paraphrase-style trajectory edits is an important direction for future work.

References

[1] Scott Aaronson. Watermarking GPT outputs. 2022.

[2] Hyeseon An, Shinwoo Park, Suyeon Woo, and Yo-Sub Han. DITTO: A Spoofing Attack Framework on Watermarked LLMs via Knowledge Distillation. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026.

[3] Anthropic. Claude Code, 2025. `https://claude.com/product/claude-code`.

[4] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *Proceedings of the 37th Conference on Learning Theory (COLT)*, 2024.

[5] Sumanth Dathathri, Abigail See, Sumedh Ghaisas, Po-Sen Huang, Rob McAdam, Johannes Welbl, Vandana Bachani, Alex Kaskasoli, Robert Stanforth, Tatiana Matejovicova, et al. Scalable watermarking for identifying large language model outputs. *Nature*, 2024.

[6] Zehang Deng, Yongjian Guo, Changzhou Han, Wanlun Ma, Junwu Xiong, Sheng Wen, and Yang Xiang. AI agents under threat: A survey of key security challenges and future pathways. *ACM Computing Surveys*, 2025.

[7] Jinyang Ding, Kejiang Chen, Yaofei Wang, Na Zhao, Weiming Zhang, and Nenghai Yu. Discop: Provably secure steganography in practice based on "distribution copies". In *2023 IEEE Symposium on Security and Privacy (SP)*, 2023.

[8] Thibaud Gloaguen, Robin Staab, Nikola Jovanović, and Martin Vechev. LLM fingerprinting via semantically conditioned watermarks. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026.

[9] Boyu Gou, Zanming Huang, Yuting Ning, Yu Gu, Michael Lin, Weijian Qi, Andrei Kopanev, Botao Yu, Bernal Jiménez Gutiérrez, Yiheng Shu, et al. Mind2Web 2: Evaluating agentic search with agent-as-a-judge. *arXiv preprint arXiv:2506.21506*, 2025.

[10] Zhiwei He, Binglin Zhou, Hongkun Hao, Aiwei Liu, Xing Wang, Zhaopeng Tu, Zhuosheng Zhang, and Rui Wang. Can watermarks survive translation? on the cross-lingual consistency of text watermark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[11] Abe Bohan Hou, Jingyu Zhang, Tianxing He, Yichen Wang, Yung-Sung Chuang, Hongwei Wang, Lingfeng Shen, Benjamin Van Durme, Daniel Khashabi, and Yulia Tsvetkov. SemStamp: A semantic watermark with paraphrastic robustness for text generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2024.

[12] Kaibo Huang, Zipei Zhang, Zhongliang Yang, and Linna Zhou. Agent guide: A simple agent behavioral watermarking framework. *arXiv preprint arXiv:2504.05871*, 2025.

[13] Kaibo Huang, Jin Tan, Yukun Wei, Wanling Li, Zipei Zhang, Hui Tian, Zhongliang Yang, and Linna Zhou. AgentMark: Utility-preserving behavioral watermarking for agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. To appear.

[14] Nikola Jovanović, Robin Staab, and Martin Vechev. Watermark stealing in large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[15] Gabriel Kaptchuk, Tushar M. Jois, Matthew Green, and Aviel D. Rubin. Meteor: Cryptographically secure steganography for realistic distributions. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.

[16] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[17] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[18] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. Robust distortion-free watermarks for language models. *Transactions on Machine Learning Research (TMLR)*, 2024.

[19] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[20] Guorui Liao, Jinshuai Yang, Weizhi Shao, and Yongfeng Huang. A framework for designing provably secure steganography. In *34th USENIX Security Symposium (USENIX Security)*, 2025.

[21] Aiwei Liu, Leyi Pan, Xuming Hu, Shiao Meng, and Lijie Wen. A semantic invariant robust watermark for large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[22] Wenlong Meng, Chen Gong, Terry Yue Zhuo, Fan Zhang, Kecen Li, Zheng Liu, Zhou Yang, Chengkun Wei, and Wenzhi Chen. Watermarking LLM agent trajectories. *arXiv preprint arXiv:2602.18700*, 2026.

[23] Microsoft. Microsoft copilot, 2023. `https://copilot.microsoft.com/`.

[24] Gustavo A Oliva, Gopi Krishnan Rajbahadur, Aaditya Bhatia, Haoxiang Zhang, Yihao Chen, Zhilong Chen, Arthur Leung, Dayi Lin, Boyuan Chen, and Ahmed E Hassan. SPICE: An automated SWE-Bench labeling pipeline for issue clarity, test coverage, and effort estimation. *arXiv preprint arXiv:2507.09108*, 2025.

[25] OpenAI. Introducing deep research, 2025. `https://openai.com/index/introducing-deep-research/`.

[26] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[27] Qi Pang, Shengyuan Hu, Wenting Zheng, and Virginia Smith. No free lunch in LLM watermarking: Trade-offs in watermarking design choices. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[28] Shinwoo Park, Hyejin Park, Hyeseon An, and Yo-Sub Han. A linguistics-aware llm watermarking via syntactic predictability. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. To appear.

[29] Shinwoo Park, Hyejin Park, Hyeseon An, and Yo-Sub Han. WaterMod: Modular Token-Rank Partitioning for Probability-Balanced LLM Watermarking. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026.

[30] Mark Russinovich, Yanan Cai, and Ahmed Salem. Hey, that's my model! introducing chain & hash, an LLM fingerprinting technique. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026.

[31] Zhensu Sun, Xiaoning Du, Fu Song, Mingze Ni, and Li Li. CoProtector: Protect open-source code against unauthorized training usage with data poisoning. In *Proceedings of the ACM Web Conference 2022 (WWW)*, 2022.

[32] Zhensu Sun, Xiaoning Du, Fu Song, and Li Li. CodeMark: Imperceptible watermarking for code datasets against neural code completion models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.

[33] Harold Triedman, Rishi Jha, and Vitaly Shmatikov. Multi-agent systems execute arbitrary malicious code. *arXiv preprint arXiv:2503.12188*, 2025.

[34] Liwen Wang, Wenxuan Wang, Shuai Wang, Zongjie Li, Zhenlan Ji, Zongyi Lyu, Daoyuan Wu, and Shing-Chi Cheung. IP leakage attacks targeting LLM-based multi-agent systems. *arXiv preprint arXiv:2505.12442*, 2025.

[35] Liwen Wang, Zongjie Li, Yuchong Xie, Shuai Wang, Dongdong She, Wei Wang, and Juergen Rahmel. On protecting agentic systems' intellectual property via watermarking. *arXiv preprint arXiv:2602.08401*, 2026.

[36] Yuan Xiao, Yuchen Chen, Shiqing Ma, Haocheng Huang, Chunrong Fang, Yanwei Chen, Weisong Sun, Yunfeng Zhu, Xiaofang Zhang, and Zhenyu Chen. DeCoMa: Detecting and purifying code dataset watermarks through dual channel code abstraction. *Proceedings of the ACM on Software Engineering (ISSTA)*, 2025.

[37] Jiashu Xu, Fei Wang, Mingyu Derek Ma, Pang Wei Koh, Chaowei Xiao, and Muhao Chen. Instructional fingerprinting of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2024.

[38] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. AgentTrek: Agent trajectory synthesis via guiding replay with web tutorials. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[39] John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2025.

[40] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[41] Kaixiang Zhao, Lincan Li, Kaize Ding, Neil Zhenqiang Gong, Yue Zhao, and Yushun Dong. A survey on model extraction attacks and defenses for large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), Volume 2*, 2025.

A Social Impact and Limitations

Reliable provenance for autonomous LLM agents is an increasingly important capability as such systems are deployed in consequential tool-use settings, and we view action-trace watermarking as net-positive for accountability and post-hoc auditing; deployers should nonetheless communicate

clearly when such schemes are active. Several natural extensions remain open and we view them as promising directions rather than shortcomings of the present construction. Stronger adversarial models, including adaptive attackers and semantic-level transformations such as paraphrasing, raise questions that any behavior-level watermark must eventually engage with, and we expect the redundant-coding perspective developed here to provide a useful starting point. Tightening the analytical characterization of multi-channel redundancy beyond the orthogonal benefits we identify (calibration concentration, robustness buffer, detector extensibility) is likewise an interesting theoretical direction.

B Full Related Work

B.1 Watermarking for Large Language Models

Watermarking has emerged as a principled mechanism for attributing content generated by large language models (LLMs). The seminal scheme of Kirchenbauer et al. [16] partitions the vocabulary into green and red lists via a keyed pseudorandom function and biases the sampling distribution toward the green list, enabling zero-bit detection with a z-statistic. Subsequent work has sought to reduce the utility or perceptual cost of biased sampling. Low-distortion and distribution-preserving designs include the Gumbel-style scheme of Aaronson [1], the cryptographically undetectable construction of Christ et al. [4], the inverse-transform sampling watermark of Kuditipudi et al. [18], and the production-ready SynthID-Text system of Dathathri et al. [5], which was evaluated in a live deployment involving nearly twenty million Gemini responses. A complementary line conditions the watermark on sentence-level semantics to improve robustness to paraphrasing and lexical rewriting [21, 11]. Cross-lingual robustness has been studied separately: translation can substantially weaken existing text watermarks, motivating cross-lingual defenses such as X-SIR [10]. Orthogonal refinements balance watermark mass across token-rank partitions [29] or adapt the green-list bias to local syntactic predictability [28].

A growing body of work probes the limits of this paradigm. Kirchenbauer et al. [17] stress-test detection under paraphrase and length variation; Jovanović et al. [14] demonstrate that current schemes admit watermark stealing, in which an adversary approximates secret watermarking rules from observed outputs and then mounts both spoofing and scrubbing attacks; and Pang et al. [27] analyze no-free-lunch trade-offs among robustness, utility or usability, and resistance to removal and spoofing attacks. More recently, An et al. [2] show that a student model trained on outputs from a watermarked victim can inherit and reuse the watermark signal, enabling spoofing without access to the victim's logits, secret key, or watermarking scheme.

A separate strand operates on the model or training data rather than only on sampled outputs. Model fingerprinting embeds keyed challenge–response associations: instructional fingerprinting uses lightweight instruction tuning so that private triggers elicit identifying responses [37], while Chain & Hash constructs hash-linked question–answer challenges for model ownership verification [30]. Semantically conditioned variants further attach the fingerprint to broad semantic domains, reducing reliance on small fixed trigger sets and making simple trigger filtering or purification less effective [8]. The closest analogues for watermarking training corpora are code dataset watermarks, which introduce data-poisoning or backdoor-style features [31] or adaptive semantics-preserving program transformations [32]; such schemes have nevertheless been shown to be detectable and removable through automated code abstraction [36]. Across these axes, however, the evidence used for verification is generated text, challenge–response behavior, code artifacts, or model-level behavior. These settings do not directly address provenance for the discrete, partially observable action trajectories produced by LLM agents.

B.2 Watermarking for LLM Agents

The schemes above generally assume that detection can inspect the relevant token sequence, challenge-response interface, or model artifact. Modern LLM agents violate this assumption. Agent execution interleaves model-generated reasoning or actions with environment-generated observations [40], and in many deployed or grey-box agentic settings, internal token streams and reasoning traces may be unavailable, leaving only visible actions and final responses as practical evidence [12, 13, 35]. Token-level signals can be diluted by non-trainable observations and are largely lost when high-level decisions are compiled into low-bandwidth tool invocations such as `bookmark` or `send_email`.

These properties have motivated a separate line of work that watermarks agent behavior rather than surface text.

Behavior-level watermarking. Huang et al. [12] introduce AGENT GUIDE, a behavioral watermarking framework for LLM agents. The scheme elicits a probability distribution over a finite list of high-level behaviors at each step and applies watermark-guided probability biases to a keyed subset, so that watermarked agents select guided behaviors more frequently than expected under the unwatermarked null. Detection is performed over multiple decision rounds using a one-sided z-test against a binomial null. Because the scheme explicitly changes per-step behavior probabilities, however, its distributional cost can accumulate over long horizons. Building on this limitation, Huang et al. [13] propose AGENTMARK, which embeds multi-bit identifiers into the same behavioral layer while preserving the elicited per-step distribution. Their concrete instantiation combines distribution recombination of behavior probabilities into uniform bins with cyclic-shift uniform encoding within each bin [15, 7, 20], together with random linear network coding for erasure-resilient recovery from partial trajectories. The two schemes therefore place the statistical signal in different locations: AGENT GUIDE concentrates evidence in marginal frequency shifts of guided behaviors and admits a simple z-test, whereas AGENTMARK preserves the per-step marginal distribution and recovers a sequence-level payload through structured decoding.

Trajectory and system-level provenance. A complementary thread targets the training data and the deployed system rather than runtime decision sampling. Meng et al. [22] address the unauthorized redistribution of expert or high-value agent trajectories [26, 39, 38, 19, 9], whose construction and validation can require substantial annotation and labeling effort [24]. Their method, ACTHOOK, injects auxiliary hook actions at ambiguous or high-entropy decision points, conditioned on a secret activation phrase appended to a small subset of training prompts. An agent fine-tuned on the watermarked corpus learns this trigger-conditioned behavior, which is then verified through a paired hypothesis test against a sham trigger. Wang et al. [35] take a third perspective and treat the entire fine-tuned agent as the proprietary asset under a grey-box visibility model in which only the action sequence and final response are observable. In this setting, AGENTWM exploits semantically equivalent action sequences by biasing the distribution over functionally identical tool execution paths, with verification performed via statistical hypothesis testing.

Beyond direct watermarking, a growing body of work documents the broader IP attack surface of agentic systems, including prompt extraction in multi-agent pipelines [34], malicious code execution in multi-agent orchestration [33], systematic surveys of agent security risks [6], and the broader model-extraction threat landscape for LLMs [41]. These threats are largely orthogonal to the watermarking question, yet they reinforce the case for auditable provenance mechanisms in regulated deployments.

C Algorithm Pseudocode

We provide explicit pseudocode for the three procedures of `SeqWM`: the multi-channel encoder (Algorithm 1), the sliding-window score computation (Algorithm 2), and the random-key calibrated detector (Algorithm 3).

D Proofs

D.1 Proof of Theorem 4.1

We prove the two parts separately. Throughout, let $b_{1:T}$ denote the clean watermarked sequence and $\hat{b}$ the result of deleting d positions chosen uniformly at random, with $\rho = d/T$.

Proof of (a): round-indexed collapse. The round-indexed encoder of Huang et al. [12] biases the action at step t toward the subset $G_{\mathcal{K}}(t)$, so for the clean sequence the indicator $\mathbb{1}[b_t \in G_{\mathcal{K}}(t)]$ has expectation $p_1 = p_0 + \Theta(\gamma p_0(1-p_0))$ under the watermarked distribution.

After deletion, let $k^* \in \{1, \dots, T\}$ denote the original position of the first deleted action. For each detector position $t < k^*$, the surviving action retains its original position index, so the round-indexed indicator $\mathbb{1}[\hat{b}_t \in G_{\mathcal{K}}(t)]$ continues to evaluate the same encoder–detector pairing as in the clean sequence. Its expectation remains p_1.

Algorithm 1 Multi-channel encoder.

Require: key $\mathcal{K}$, horizon T, window w, channels m, subset size n, bias γ

history $\leftarrow [\,]$
for $t = 1, \ldots, T$ **do**
  $(\mathcal{B}_t, P_t) \leftarrow \textsc{ElicitFromLLM}(\text{state}_t)$
  if $|\text{history}| < w$ **then**
    $b_t \sim P_t$ ▷ bootstrap step: no watermark
  else
    $c_t \leftarrow \text{history}[-w:]$
    $B_{g,t}^{(j)} \leftarrow G_{\mathcal{K}}(c_t \,\|\, j)$ for $j = 1, \ldots, m$
    Compute $s_t(b)$ and $P_t^{\text{wm}}(b)$ for all $b \in \mathcal{B}_t$ via Eqs. (4)–(5)
    $b_t \sim P_t^{\text{wm}}$
  end if
  $\text{history}.\textsc{append}(b_t)$
end for
return $b_{1:T}$

Algorithm 2 Sliding-window score computation.

Require: key $\mathcal{K}$, observed sequence $\hat{b}_{1:T'}$, parameters (w, m, n)

$S \leftarrow 0$
for $i = 1, \ldots, T' - w$ **do**
  $c \leftarrow \hat{b}_{i:i+w-1}, \quad b_{\text{next}} \leftarrow \hat{b}_{i+w}$
  for $j = 1, \ldots, m$ **do**
    $B_g \leftarrow G_{\mathcal{K}}(c \,\|\, j)$
    if $b_{\text{next}} \in B_g$ **then** $S \leftarrow S + 1$
    end if
  end for
end for
return S

Algorithm 3 Random-key calibrated detection.

Require: key $\mathcal{K}$, observation $\hat{b}_{1:T'}$, parameters (w, m, n), calibration size M, level α

$S_{\text{true}} \leftarrow S(\mathcal{K}, \hat{b}_{1:T'})$ ▷ Eq. (9); via Algorithm 2
$\mathcal{S}_{\text{null}} \leftarrow [\,]$
for $r = 1, \ldots, M$ **do**
  $\mathcal{K}_r \sim \text{Uniform}(\text{KeySpace})$
  $\mathcal{S}_{\text{null}}.\textsc{append}\big(S(\mathcal{K}_r, \hat{b}_{1:T'})\big)$
end for
$p \leftarrow \big(1 + \big|\{S_r \in \mathcal{S}_{\text{null}} : S_r \geq S_{\text{true}}\}\big|\big)/(M+1)$
return $\mathbb{1}[p < \alpha],\ p$

For each detector position $t \geq k^*$ that exists in the deleted sequence, however, the action at detector position t originated from a strictly later encoder position $t' > t$ in the clean sequence. The encoder used $G_{\mathcal{K}}(t')$ to bias this action, but the detector evaluates it against $G_{\mathcal{K}}(t)$. By the pseudorandom misalignment idealization stated in the theorem, this off-index comparison contributes only the null hit rate:

$$\mathbb{E}\big[\mathbb{1}[\hat{b}_t \in G_{\mathcal{K}}(t)]\big] \;=\; p_0, \qquad t \geq k^*. \tag{13}$$

Summing across positions, the expected score gap relative to a non-watermarked sequence with expectation p_0 at every aligned position is

$$\mathbb{E}\big[S_{\text{after}}^{\text{RI}} - S_{\text{null}}^{\text{RI}} \,\big|\, k^*\big] \;=\; (k^* - 1) \cdot (p_1 - p_0) \;=\; (k^* - 1) \cdot \Theta(\gamma p_0 (1 - p_0)). \tag{14}$$

Under uniform random deletion, k^* is the minimum of d uniformly chosen positions in $\{1, \ldots, T\}$. A standard computation gives $\mathbb{E}[k^*] = (T+1)/(d+1)$, and hence $\mathbb{E}[k^* - 1] = (T-d)/(d+1) =$

$O(1/\rho)$ for any fixed $\rho > 0$. Therefore,

$$\mathbb{E}\big[S^{\mathrm{RI}}_{\mathrm{after}} - S^{\mathrm{RI}}_{\mathrm{null}}\big] \;=\; \frac{T-d}{d+1} \cdot \Theta(\gamma p_0(1-p_0)) \;=\; O(1/\rho). \tag{15}$$

The clean-sequence gap is $\mathbb{E}[S^{\mathrm{RI}}_{\mathrm{clean}} - S^{\mathrm{RI}}_{\mathrm{null}}] = T \cdot \Theta(\gamma p_0(1-p_0)) = \Theta(T)$. The ratio of post-attack gap to clean gap is therefore $O(1/(\rho T))$, which vanishes as $T \to \infty$ for any fixed $\rho > 0$. □

Proof of (b): sliding-window deterministic bound. Recall the score

$$S(\mathcal{K}, b_{1:T}) \;=\; \sum_{i=1}^{T-w} \sum_{j=1}^{m} \mathbb{1}[b_{i+w} \in G_{\mathcal{K}}(b_{i:i+w-1} \,\|\, j)]\,.$$

Each term in the outer sum is indexed by a pair (c, b^+) where $c = b_{i:i+w-1}$ is a length-w context and $b^+ = b_{i+w}$ is its immediate successor; we refer to such a pair as a *contextual transition*. The sequence $b_{1:T}$ contains exactly $T - w$ contextual transitions, and each contributes m indicators to S.

Fix any deleted position $k \in \{1, \ldots, T\}$. We claim that b_k participates in at most $w + 1$ contextual transitions of the clean sequence. Indeed, b_k appears in the context $b_{i:i+w-1}$ if and only if $i \le k \le i + w - 1$, i.e., $i \in \{k - w + 1, \ldots, k\}$, giving at most w such contexts. Additionally, b_k appears as the successor b_{i+w} if and only if $i + w = k$, giving exactly one such transition (when $k > w$). Summing, b_k participates in at most $w + 1$ contextual transitions and therefore contributes to at most $(w + 1)\, m$ indicators in $S(\mathcal{K}, b_{1:T})$.

Deleting b_k may invalidate at most the clean indicators in which b_k participates; all other clean indicators are computed from contexts and successors that exclude position k and remain present, possibly at shifted indices. Each invalidated indicator can change by at most 1 in absolute value, and we are interested in the worst-case decrease, which is at most 1 per indicator. Hence

$$S(\mathcal{K}, b_{1:T} \setminus \{b_k\}) \;\ge\; S(\mathcal{K}, b_{1:T}) - (w+1)\, m.$$

For d deletions, iterating this argument yields

$$S(\mathcal{K}, \hat{b}) \ge S(\mathcal{K}, b_{1:T}) - d(w+1)m.$$

This proves the deterministic additive bound. We do not convert this into a universal multiplicative bound on the detection gap, since the clean score and the null-calibrated score gap depend on the action distribution and the watermark bias. □

E Experimental Setup Details

E.1 Models and Inference

We use three open-weight instruction-tuned LLMs as the agent backbone, covering three families and two parameter scales:

Model	HuggingFace ID
Llama-3.2-3B	`meta-llama/Llama-3.2-3B-Instruct`
Gemma-4-E4B	`google/gemma-4-E4B-it`
Qwen3-4B	`Qwen/Qwen3-4B-Instruct-2507`

All models are loaded in bfloat16 via `transformers==5.7.0` on Python 3.10. The decoding temperature for the agent LLM is fixed at 0.7 across all experiments. We use a single agent profile, `active_calm` (the Active Calm persona from the Agent Guide profile set), throughout to isolate watermark effects from persona-induced variance.

E.2 Datasets

We evaluate on three agent benchmarks that span tool-use, embodied reasoning, and social simulation:

Dataset	Scale	Episode setting
ToolBench	200 queries (50 for γ-sweep)	ReAct loop, `max_steps_per_episode` $= 6$
ALFWorld	50 tasks	`max_episode_length` $= 50$, `num_few_shot` $= 1$
OASIS Reddit	100 simulation steps	1 user agent + 3 NPC agents, persona `tech_geek`

All three benchmarks share the same agent profile (`active_calm`) and the same watermark plumbing. Per-step behavior sets are dataset-specific: ToolBench surfaces the candidate tool list at each ReAct step, ALFWorld exposes the admissible-action list from the textual environment, and OASIS Reddit uses a fixed action vocabulary of social actions (e.g., create post, like, comment, follow).

E.3 Watermark Hyperparameters

Unless otherwise stated, all experiments use the following configuration:

Symbol	Value	Meaning
w (`window_size`)	3	History window for context-conditioned seeding
n (`n_guide`)	3 (OASIS: 2)	Size of each guided subset $B_g^{(j)}$
`n_min`	2 (OASIS: 1)	Minimum subset size after clipping
m (`m_channels`)	8	Number of redundant channels
γ (`gamma_mc`)	2.0	Bias strength in $P^{wm} \propto \max(P, \delta) \cdot \exp(\gamma \cdot s/m)$
δ (`delta`)	0.2	Floor applied to P before the exponential bias
M (`M_calibration`)	1000	Random-key calibration trials per detection
α (`detection_alpha`)	0.01	p-value threshold for the headline detection result
z-threshold	2.0	Threshold for the reference single-channel z-test

OASIS Reddit uses smaller n and `n_min` because its action vocabulary is narrower than ToolBench's per-step tool list or ALFWorld's per-step admissible-action set. All other parameters are shared across datasets to keep the comparison clean.

E.4 γ Sensitivity Sweep

For the γ-sensitivity experiment we sweep $\gamma \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ with $m_{\text{channels}} = 8$ fixed. Each γ value runs on a 50-query ToolBench subset (`configs/sweep_react.json`), generating both the no-watermark baseline trajectory and the multi-channel watermarked trajectory from the same query set per worker.

E.5 Computational Resources

All experiments run on a SLURM-managed cluster with two relevant GPU partitions:

- `suma_a100`: $2\times$ NVIDIA A100 GPUs, used for the main rollout experiments (main detection table, ablations, γ-sweep, cross-model runs).
- `suma_pro6000`: $2\times$ NVIDIA RTX PRO 6000 Blackwell GPUs, used for sanity checks and post-hoc analysis sweeps (such as the deletion-attack re-detection, which does not require LLM inference).

A single full main run (200 ToolBench queries, all modes side by side under one rollout) completes in roughly 4 to 6 hours of A100 wall time per agent LLM.